

CONSTITUTIONAL AGENTS

EN PRODUCCIÓN

ASTRO V7.3.2 SSR

CLOUDFLARE WORKERS EDGE

D1 SQLITE + FTS5

VECTORIZE RAG

LLAMA 4 & BGE-M3

OPENAPI 3.1 & SWAGGER UI

TYPESCRIPT & DRIZZLE

Constitutional Agents

Orquestador Serverless en el Edge de Agentes de Inteligencia Artificial especializados en Derecho Constitucional Colombiano (Constitución Política de 1991).

REPOSITORIO OFICIAL EN GITHUB
github.com/SantiagoPrada2005/constitution
al-agents

DESPLIEGUE EN PRODUCCIÓN (EDGE)
proyecto-agentes-ia.santiagopradamoreno.w
orkers.dev

CONSOLA SWAGGER UI INTERACTIVA
...workers.dev/swagger (/docs)

📌 1. Descripción y Propósito del Proyecto

Constitutional Agents es una plataforma integral de gestión (CRUD), orquestación y consulta conversacional con **Generación Aumentada por Recuperación (RAG)** de agentes jurídicos autónomos. El sistema modela agentes cognitivos con instrucciones parametrizables, catálogo de habilidades normativas y un corpus constitucional estructurado que comprende los Principios Fundamentales, Derechos y Garantías (individuales, sociales, colectivos), Mecanismos de Participación y la Acción de Tutela.

Desarrollado bajo principios de *Clean Architecture*, opera al 100% en el borde (Edge Computing) sobre el ecosistema nativo de Cloudflare y Astro SSR, garantizando latencias mínimas en milisegundos, alta disponibilidad global y costo operativo cero en reposo.

⚖️ **Asesoría Jurídica Taxativa**

Responde consultas legales con estricta sujeción a la Constitución de 1991, citando expresamente artículos, principios y garantías aplicables.

📄 **Ingesta y Segmentación**

Chunking semántico de corpus normativo en Markdown por títulos y artículos, sincronizando índices vectoriales y relacionales en tiempo real.

🔄 **Pipeline RAG Híbrido**

Estrategia dual: recuperación vectorial semántica con Cloudflare Vectorize combinada con búsqueda léxica SQLite FTS5 en Cloudflare D1.

🏗️ 2. Arquitectura del Sistema (Edge Native)

La solución implementa una arquitectura serverless desacoplada en tres dimensiones independientes, eliminando la necesidad de servidores dedicados o contenedores persistentes:

CAPA ARQUITECTÓNICA	TECNOLOGÍAS CLAVE	RESPONSABILIDADES Y PATRONES DE DISEÑO
1. Presentación (UI Islands)	Astro v7.3 SSR, Tailwind CSS v4, TypeScript, Swagger UI	Renderizado en Worker (<code>output: 'server'</code>). Vistas para Dashboard de Agentes (<code>/</code>), Formulario de Gestión (<code>/nuevo</code>), Consola de Chat RAG (<code>/chat/[id]</code>) y Consola Interactiva Swagger UI (<code>/swagger</code> con alias <code>/docs</code>). Cliente HTTP tipado (<code>ApiClient</code>).
2. Dominio y Controladores	Astro Endpoints, Zod v4, Drizzle-Zod, OpenAPI 3.1	API REST modular (<code>/api/agentes</code> , <code>/api/habilidades</code>). Especificación técnica estandarizada (<code>/api/openapi.json</code> y <code>/api/swagger.json</code>). Validación en runtime con esquemas Zod. <code>AgentService</code> para reglas de negocio y <code>RagService</code> para orquestación RAG y fallback determinista.
3. Persistencia y Datos Edge	Cloudflare D1, Vectorize, Workers AI, Drizzle ORM	Patrón Repositorio (<code>AgentRepository</code> , <code>DocumentRepository</code>). Base de datos relacional SQLite con triggers FTS5 e índice vectorial con distancia coseno para búsqueda semántica de alta fidelidad.

💡 **Desacoplamiento y Portabilidad:** Las entidades del dominio y los repositorios están completamente aislados de los controladores HTTP. La sustitución del motor de inferencia o de almacenamiento relacional no impacta las reglas de negocio.

3. Pipeline RAG Constitucional Híbrido

El sistema mitiga eficazmente las alucinaciones en modelos de lenguaje mediante una estrategia de recuperación híbrida (semántica + léxica) y limitación estricta del contexto normativo:

- Ingesta y Chunking Semántico:** La función `splitMarkdownIntoSections` fragmenta el corpus constitucional respetando jerarquía de títulos y artículos. Cada sección es vectorizada usando el modelo de embeddings `@cf/baai/bge-m3` en Cloudflare Workers AI e indexada en Cloudflare Vectorize con metadatos (`agente_id`, `titulo_seccion`, `nombre_archivo`).
- Persistencia Dual y Triggers FTS5:** El texto del fragmento se persiste en la tabla relacional `agente_documentos` en Cloudflare D1. Mediante tres triggers SQL automáticos (`trg_documentos_insert`, `update`, `delete`), la tabla virtual de búsqueda de texto completo `fts_documentos` se mantiene sincronizada sin intervención manual.
- Recuperación Híbrida (`retrieveContext`):**
 - Paso Semántico:** Inferencia de embeddings sobre la consulta del ciudadano y extracción de los fragmentos más afines (`topK: 3`) en Vectorize filtrados por el agente.
 - Paso Léxico:** Consulta de texto completo SQLite FTS5 en D1 para capturar menciones normativas literales (ej. "Artículo 86", "Acción de Tutela", "Habeas Data").
 - Fusión y Deduplicación:** Consolidación de candidatos suprimiendo redundancias y proveyendo inyección de respaldo determinista ante consultas atípicas.
- Inferencia Jurídica Fundamentada:** Generación del dictamen mediante `@cf/meta/llama-4-scout-17b-16e-instruct` a baja temperatura (`0.2`), gobernado por un system prompt que prohíbe emitir conceptos sin cita taxativa del marco normativo provisto.

4. Modelo de Datos Relacional y Búsqueda Léxica

Esquema relacional implementado en Cloudflare D1 (SQLite) con integridad referencial, borrado en cascada y tabla virtual FTS5:

```
agentes (id PK, slug UNIQUE, nombre, rol, modelo, temperatura, system_prompt, activo, created_at)
habilidades (id PK, codigo UNIQUE, nombre, descripcion)
agente_habilidades (agente_id FK -> agentes, habilidad_id FK -> habilidades, PK(agente_id, habilidad_id))
agente_documentos (id PK, agente_id FK -> agentes, vector_id, nombre_archivo, titulo_seccion, contenido)
fts_documentos USING fts5 (documento_id UNINDEXED, agente_id UNINDEXED, titulo_seccion, contenido)
```

5. Matriz de Endpoints de la API REST y Swagger

MÉTODO	ENDPOINT	ACCIÓN Y DESCRIPCIÓN TÉCNICA	CÓDIGOS HTTP
GET	/api/agentes	Listar todos los agentes registrados junto con sus habilidades asociadas	200
POST	/api/agentes	Registrar un nuevo agente, validar con Zod y vincular habilidades N:M	201, 400, 409
GET	/api/agentes/:id	Obtener detalle del agente, habilidades y catálogo de documentos indexados	200, 404
PUT	/api/agentes/:id	Actualizar configuración, parámetros LLM, prompts o habilidades	200, 400, 404
DELETE	/api/agentes/:id	Eliminación en cascada del agente, sus relaciones y fragmentos asociados	200, 404
POST	/api/agentes/:id/conocimiento	Ingstar documento Markdown: fragmentación semántica y vectorización	201, 400, 404
POST	/api/agentes/:id/consultar	Ejecutar pipeline RAG híbrido conversacional contra el agente	200, 400, 404
GET	/api/habilidades	Consultar el catálogo maestro global de habilidades disponibles	200
GET	/api/openapi.json	Especificación técnica OpenAPI 3.1 completa en formato JSON (contratos y DTOs)	200
GET	/api/swagger.json	Alias estándar de la especificación Swagger OpenAPI para herramientas cliente	200
GET	/swagger (/docs)	Consola interactiva Swagger UI para ejecución de pruebas en vivo ("Try it out")	200, 302

6. Stack Tecnológico y Versiones Homologadas

TECNOLOGÍA	VERSIÓN	ROL Y JUSTIFICACIÓN ARQUITECTÓNICA
Astro	v7.3.2	Framework SSR de alto rendimiento con arquitectura de islas y salida de servidor en el Edge.
@astrojs/cloudflare	v14.3.1	Adaptador oficial para Cloudflare Workers con compatibilidad Node.js 2024-09-23 .
Cloudflare Wrangler	v4.132.0	Herramienta CLI para despliegue, emulación y gestión de infraestructura como código.
Drizzle ORM & Kit	v0.45.2 / v0.31.10	ORM TypeScript tipado de baja sobrecarga para Cloudflare D1 y gestión de migraciones DDL.
Tailwind CSS & Vite	v4.3.3	Motor de estilos de última generación integrado con Vite vía Lightning CSS sin dependencias obsoletas.
Swagger UI & OpenAPI	v5.18.3 / v3.1.0	Especificación OpenAPI 3.1 exhaustiva y consola interactiva Swagger UI embebida ("Try it out").
Zod & Drizzle-Zod	v4.6.5 / v0.8.3	Validación tipada estricta en tiempo de ejecución para esquemas de entrada y contratos REST.
TypeScript	v6.0.3	Compilador estricto para garantizar solidez de tipos desde el cliente hasta la base de datos.
pnpm	v11.x (Mandatorio)	Gestor de paquetes exclusivo del workspace que garantiza lockfiles inmutables y deduplicación.

7. Guía de Instalación y Puesta en Marcha

Pasos para clonar el repositorio, instalar dependencias con pnpm (estándar mandatorio) y ejecutar el entorno:

```
# 1. Clonar el repositorio oficial de GitHub
git clone https://github.com/SantiagoPrada2005/constitutional-agents.git
cd constitutional-agents

# 2. Instalar dependencias mediante pnpm (estándar exclusivo)
pnpm install

# 3. Aplicar migraciones del esquema y semillero de datos iniciales en Cloudflare D1
pnpm db:migrate
pnpm db:seed

# 4. Iniciar el entorno de desarrollo local en el puerto estándar
pnpm dev
# Aplicación interactiva: http://localhost:4321 • Consola Swagger UI: /swagger

# 5. Ejecutar la suite automatizada de pruebas HTTP
pnpm test
```

8. Accesos Oficiales del Proyecto

 REPOSITORIO DE CÓDIGO
github.com/SantiagoPrada2005/constitutional-agents

 DESPLIEGUE EN PRODUCCIÓN
proyecto-agentes-ia.santiagopradamoreno.workers.dev

 CONSOLA SWAGGER UI
[workers.dev/swagger \(/docs\)](https://workers.dev/swagger (/docs))

9. Autores, Colaboradores y Licencia de Uso

Desarrollado por: Santiago Prada • **GitHub:** @SantiagoPrada2005 • **Email:** santiagopradamoreno@gmail.com
Rol: Arquitectura de software, orquestación serverless en el Edge, pipeline RAG híbrido y persistencia D1.

Colaboradora: Dana Sofía Arias • **Email:** sofiaamontes101@gmail.com
Contribución: Especificación técnica OpenAPI 3.1.0, endpoints JSON y consola interactiva de documentación Swagger UI.

Licencia: Este proyecto se distribuye bajo los términos de la **Licencia MIT**. Código abierto y disponible para auditoría y reutilización académica y profesional.